
InfiniPot-V: Memory-Constrained KV Cache Compression for Streaming Video Understanding

Minsoo Kim^{1†} Kyuhong Shim² Jungwook Choi^{1‡} Simyung Chang^{3‡}

¹Hanyang University ²Sungkyunkwan University

³Qualcomm AI Research, Qualcomm Korea YH[§]

{minsoo2333, choij}@hanyang.ac.kr khshim@skku.edu

simychan@qti.qualcomm.com

Abstract

Modern multimodal large language models (MLLMs) can reason over hour-long video, yet their key–value (KV) cache grows linearly with time—quickly exceeding the fixed memory of phones, AR glasses, and edge robots. Prior compression schemes either assume the whole video and user query are available offline or must first build the full cache, so memory still scales with stream length. InfiniPot-V is the first training-free, query-agnostic framework that enforces a hard, length-independent memory cap for *streaming* video understanding. During video encoding it monitors the cache and, once a user-set threshold is reached, runs a lightweight compression pass that (i) removes temporally redundant tokens via Temporal-axis Redundancy (TaR) metric and (ii) keeps semantically significant tokens via Value-Norm (VaN) ranking. Across four open-source MLLMs and four long-video and streaming-video benchmarks, InfiniPot-V cuts peak GPU memory by up to 94%, sustains real-time generation, and matches or surpasses full-cache accuracy—even in multi-turn dialogues. By dissolving the KV cache bottleneck without retraining or query knowledge, InfiniPot-V closes the gap for on-device streaming video assistants.

1 Introducción

Los avances recientes en multimodal large language models (MLLMs) han ampliado drásticamente el alcance del razonamiento visual. El vision–language instruction tuning ahora permite que un único backbone responda preguntas abiertas sobre secuencias de video largas [27, 30, 48], mientras que técnicas de extensión de contexto como FlashAttention-2 y RingAttention llevan la ventana efectiva al régimen del millón de tokens [7, 26, 33]. Estos avances sustentan una nueva generación de *asistentes de video en streaming* y *robots humanoides* que prometen comprensión continua y en tiempo real de la escena en teléfonos móviles, gafas AR y robots edge [14, 31, 42, 37].

Streaming video understanding (SVU) difiere de la comprensión convencional de video offline (OVU). Los modelos offline ven el clip completo y la consulta del usuario antes de la inferencia, por lo que pueden adaptar cada paso de compresión o recuperación. En streaming, los frames llegan de manera incremental y las consultas futuras son desconocidas, lo que obliga a que todo el procesamiento previo a la consulta sea *query-agnostic*. Además, la memoria del dispositivo es fija, pero el Transformer emite cientos de tokens por frame, por lo que la caché key–value (KV) crece linealmente. Por ejemplo, un clip de 15 minutos a 10 fps procesado por LLaVA-Next-Video-7B ya requiere ~ 100 GB

[†]Work done during an internship at Qualcomm Technologies, Inc.

[‡]Corresponding authors.

[§]Qualcomm AI Research, an initiative of Qualcomm Technologies, Inc.

de almacenamiento KV, muy por encima de las decenas de gigabytes disponibles en plataformas móviles o robóticas [52, 19].

Los trabajos previos abordan las restricciones de memoria en videos largos en tres etapas (Fig.1). *Frame Sampling* [10] descarta frames antes de la codificación, reduciendo memoria pero degradando severamente la cobertura temporal y la precisión. *Input-Vision Compression (IVC)* [36, 40] elimina vision tokens redundantes después de la codificación, reduciendo la carga de Prefill pero requiriendo todavía que el conjunto completo de vision tokens se almacene en memoria. *KV cache Compression (KVC)* [24, 12] selecciona tokens relevantes para la consulta después del paso de Prefill, ofreciendo la mayor precisión, pero solo después de materializar la caché KV completa. El desafío se intensifica en escenarios de streaming: el uso de memoria de Frame Sampling, IVC y KVC crece casi linealmente con la longitud del video, hasta superar los límites del dispositivo. El KV cache offloading (por ejemplo, ReKV [35]) amplía el espacio de memoria, pero incurre en costosas transferencias de datos repetidas para cada consulta. Por lo tanto, ningún enfoque existente ofrece la propiedad clave que SVU necesita: una compresión de video en streaming *independiente de la longitud y query-agnostic*.

Un enfoque natural para abordar las restricciones de memoria en streaming video es explotar la fuerte *redundancia espaciotemporal* de los flujos de video. Introducimos **InfiniPot-V**, el primer framework diseñado específicamente para SVU con memoria restringida. InfiniPot-V es *training-free, query-agnostic* y opera de forma *continua* durante la inferencia. Cuando la caché KV alcanza un umbral de memoria definido por el usuario M , realiza una compresión in-place que libera espacio para nuevos frames preservando la esencia semántica del contexto previo. Esta compresión está guiada por dos métricas livianas y complementarias. *Temporal-axis Redundancy (TaR)* modela los Key embeddings como un tensor 3D en el tiempo y elimina tokens con alta similitud coseno con frames recientes, podando de forma efectiva contenido estático o repetitivo. *Value-Norm Importance (VaN)* ordena los tokens restantes según la norma ℓ_2 de sus vectores Value —un fuerte proxy agnóstico al modelo de saliencia semántica— y aplica una estrategia de pooling adaptativa por capa. Esta compresión es altamente eficiente, añade una latencia despreciable y a la vez hace cumplir estrictamente los límites de memoria.

Una evaluación extensa confirma la efectividad de este diseño. En cuatro MLLMs open-source (3B y 7B) y seis benchmarks de video largo —que cubren tanto tareas offline (VideoMME, EgoSchema, MLVU, LongVideoBench) como de streaming (RVS-Ego/Movie, OVO-Bench, StreamingBench)— InfiniPot-V reduce el uso de longitud de contexto de entrada hasta 6K para contextos de 50K tokens, con una precisión que iguala o supera a las líneas base de caché completa. Mantiene rendimiento en tiempo real a 14 frames por segundo con solo 0.5% de overhead de compresión. Además, su naturaleza query-agnostic ofrece beneficios claros en configuraciones de diálogo multi-turno (Appendix. C). Al eliminar el cuello de botella de la caché KV sin reentrenamiento ni dependencia de la consulta, InfiniPot-V allana el camino para asistentes multimodales prácticos ejecutados en dispositivo.

2 Antecedentes

Buscamos desplegar aplicaciones de streaming video understanding (SVU) [49, 35] en entornos con memoria restringida. A diferencia de offline video understanding (OVU) [54, 11, 45], que asume acceso al video completo, SVU debe procesar flujos de video arbitrariamente largos y responder preguntas en cualquier instante usando solo los frames observados hasta ese momento. Dado un flujo de video $V_T := [v_1, v_2, \dots, v_T]$ con T frames y un conjunto de preguntas $Q = q_1, q_2, \dots, q_N$, SVU responde cada pregunta q_i en el tiempo t ($1 \leq t \leq T$) usando únicamente los frames observados $V_t := [v_1, v_2, \dots, v_t]$.

Como SVU trabaja con flujos de video no acotados, el procesamiento eficiente en memoria es esencial. En esta sección describimos cómo los multimodal large language models (MLLMs) manejan videos largos, revisamos enfoques previos para reducir memoria en OVU y analizamos sus limitaciones al aplicarlos a SVU. (Véase el Apéndice. F para una discusión detallada del trabajo relacionado.)

2.1 Preliminar: Offline Long Video Understanding

Procesamiento de video en MLLMs. Los Multimodal Large Language Models (MLLMs) [52, 48, 43] procesan videos offline mediante una tubería estructurada (Fig. 1(a)). Dado un video

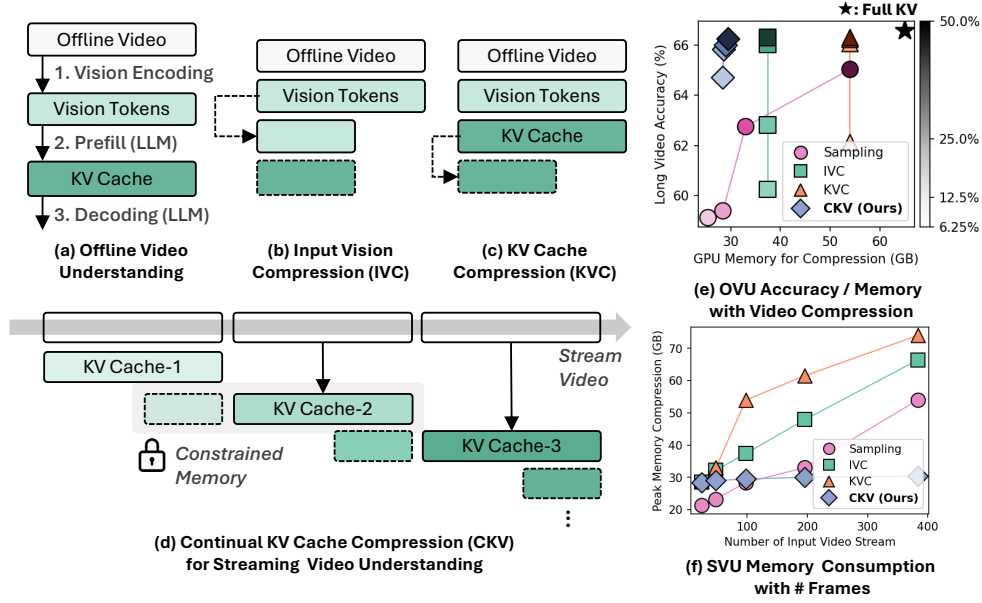


Figure 1: **Video Understanding y compresión en MLLMs.** (a) tubería OVU; (b) IVC: comprime vision tokens después del encoding; (c) KVC: comprime la KV cache después del prefill; (d) CKV: procesa y comprime iterativamente KV caches para restringir el uso de memoria; (e) Exactitud vs. consumo de memoria GPU para la compresión en cuatro razones de reducción de tokens (50%, 25%, 12.5%, 6.25%) sobre MLVU usando Qwen-2-VL-7B. LongVU[36] se usa para IVC y SnapKV[24] para KVC; (f) uso de memoria GPU a medida que aumenta la longitud del flujo de video de entrada. IVC/KVC/CKV apuntan a una cache de 6K; Sampling usa 1/4 de los frames de entrada. Medido en una única GPU A100 de 80GB.

$V_T := [v_1, v_2, \dots, v_T]$ de T frames muestreados uniformemente, un vision encoder f_{ViT} transforma cada frame en visual tokens:

$$X = f_{ViT}(V_T) = [x_1, x_2, \dots, x_N] \in \mathbb{R}^{N \times D}, \quad (1)$$

donde $N = P \times T$ denota el número total de tokens muestreados, donde P es el número de tokens por frame (determinado por la resolución de entrada y el tamaño de patch de ViT), y D es la dimensión del embedding del token.

La secuencia de tokens X se pasa luego al LLM en dos fases: Prefill y Decoding. Durante la fase de Prefill (Fig. 1(a), paso 2), todos los tokens se procesan de una vez para construir la KV cache inicial. La operación de atención calcula:

$$Q = XW_q, \quad K = XW_k, \quad V = XW_v, \quad O^{\text{attn}} = \text{Softmax}\left(\frac{QK^\top}{\sqrt{D}} + M\right)V, \quad (2)$$

donde $W_q, W_k, W_v \in \mathbb{R}^{D \times D}$ son matrices de proyección y M es una máscara causal que impone decoding autorregresivo.

En la fase de Decoding (Fig. 1(a), paso 3), el modelo genera tokens uno por uno usando keys y values almacenados de la fase de prefill. Para evitar cómputo redundante, la KV cache $\mathcal{C} = (K, V)$ se actualiza de forma incremental:

$$\mathcal{C}_{t+1} = (\{K, k_{t+1}\}, \{V, v_{t+1}\}), \quad (3)$$

donde k_{t+1}, v_{t+1} corresponden a los embeddings KV del token recién procesado.

2.2 Estrategias de compresión offline para video largo

Los videos largos producen secuencias de tokens X extremadamente extensas, lo que genera requisitos prohibitivos de memoria GPU y latencia durante el decoding. Trabajos previos abordan este cuello de botella en el entorno offline mediante tres clases de métodos (Fig. 1a-c):

(1) **Frame Sampling** [10]. Muestrear uniformemente un clip más corto $V_{T'} \subseteq V_T$ reduce la longitud de entrada y, en consecuencia, el uso de memoria también se reduce en proporción a la tasa de compresión.

(2) **Input-Vision Compression (IVC)** [36, 40]. Después del vision encoding, IVC poda agresivamente los vision tokens redundantes y conserva solo un subconjunto saliente $X' \subseteq X$ (Fig. 1b) para reducir el contexto entregado al language decoder y así realizar un Prefill comprimido en memoria.

(3) **KV cache Compression (KVC)** [24, 12, 4]. Realiza la compresión después del prefill: KVC calcula puntajes de importancia $u_t = \sum_{i=N-w}^N \text{Attn}(x_i \rightarrow x_t)$ sobre los últimos w tokens y conserva las entradas top- M para el presupuesto de memoria M aplicando la política de expulsión π , produciendo una cache comprimida $C' = \pi(C)$ para un Decoding comprimido en memoria. (Fig. 1c). Nótese que la política de expulsión π depende fuertemente del contenido de los últimos w tokens, reflejando la consulta del usuario, y por ello se la denomina método de compresión de cache dependiente de la consulta (véase el Apéndice. D para más análisis).

Estas técnicas son efectivas cuando el video completo está disponible de antemano, pero asumen implícitamente (i) memoria no restringida para la compresión y (ii) una consulta conocida o fácilmente aproximable.

2.3 Desafíos en Streaming Video Understanding

La Fig. 1(e) compara tres métodos de compresión *offline* sobre un video fijo de 50 K tokens en cuatro razones de compresión (los tonos más oscuros indican razones más altas: 50%, 25%, 12.5%, 6.25%), revelando un trade-off fundamental entre uso de memoria y exactitud. *Frame sampling* omite frames para ahorrar memoria, pero degrada severamente la exactitud de reconocimiento. Aumentar la tasa de muestreo mejora la exactitud, pero infla rápidamente el uso de memoria. *IVC* comienza con una gran huella de memoria para todos los vision tokens antes de seleccionar cuáles descartar. *KVC*, que opera sobre características key-value más expresivas, logra la mayor exactitud pero requiere la mayor cache de Prefill. Notablemente, incluso bajo un escenario *offline* favorable—con acceso completo al video y una consulta *offline*—ninguno de los métodos logra simultáneamente alta exactitud y bajo uso de memoria.

Este trade-off se vuelve más severo en el escenario de streaming video understanding (SVU). Como se muestra en la Fig. 1(f), el pico de uso de memoria GPU aumenta con la longitud del flujo. *KVC* exhibe un crecimiento de memoria casi lineal, ya que debe materializar todos los vision tokens y construir la KV cache completa antes de comprimir. Además, debido a su naturaleza dependiente de la consulta, *KVC* debe *re-ejecutar* la etapa de prefill intensiva en memoria cada vez que cambia la consulta del usuario. *Frame sampling* e *IVC* también crecen linealmente, aunque más lentamente, y terminan superando la capacidad de memoria de dispositivos edge prácticos (por ejemplo, 32GB[19]) a medida que el flujo continúa. *ReKV* [35], un método *KVC* reciente, aborda esto descargando la KV cache a memoria CPU, pero introduce una sobrecarga considerable de offloading y latencia de compresión.

Estos hallazgos resaltan dos requisitos centrales para SVU: (1) un presupuesto de memoria fijo que no crezca con la longitud del flujo, y (2) estrategias de retención de tokens independientes de la consulta. Los métodos existentes no cumplen al menos uno de estos requisitos, lo que limita su idoneidad para SVU. Para superar esto, proponemos Continual KV cache compression (CKV), ilustrado en la Fig. 1(d). CKV procesa frames en bloques pequeños y comprime la cache cada vez que se alcanza el límite fijo de memoria, garantizando un uso constante de memoria durante todo el streaming. Además, para la retención de tokens independiente de la consulta, nuestro enfoque emplea métricas espaciotemporales ligeras para identificar y preservar tokens semánticamente significativos sin depender de consultas futuras. Como resultado, a pesar de operar bajo restricciones estrictas de memoria, CKV logra una exactitud comparable o superior a *KVC* (Fig. 1(e)), mientras consume mucha menos memoria que *IVC* o *frame sampling* (Fig. 1(f)). Los detalles algorítmicos se describen en la Sec. 3.

3 InfiniPot-V: Memory-Constrained Streaming Video Understanding

Presentamos **InfiniPot-V**, un framework CKV diseñado para SVU con restricciones de memoria. Como se muestra en la Fig. 1(d) y en el Algoritmo 1, **InfiniPot-V** procesa flujos de video aplicando

Algorithm 1 Compresión continua de KV cache (CKV) con InfiniPot-V

Require: Memory budget $|M|$, target cache size $|C|$, TaR ratio α

Initialize $K, V \leftarrow \emptyset$ $\triangleright$ Empty KV cache

while el flujo de video continúa **do**

 (1) Process: $K_{\text{new}}, V_{\text{new}} \leftarrow$ Process new frame; $K \leftarrow [K; K_{\text{new}}], V \leftarrow [V; V_{\text{new}}]$ $\triangleright$ Append new tokens

if $\text{len}(K) \geq |M|$ **then** $\triangleright$ Memory budget exceeded

 (2) Extract: $K_{\text{recent}}, V_{\text{recent}} \leftarrow$ los r frames más recientes de K, V $\triangleright \text{len}(K) = \text{len}(V) = |M|$

 (3) TaR: $s^{\text{TaR}} \leftarrow \text{ComputeTaRScores}(K); \mathcal{I}_{\text{TaR}} \leftarrow \text{TopK}(s^{\text{TaR}}, \alpha|C| - \text{len}(K_{\text{recent}}))$ $\triangleright$ Sec. 3.1

 (4) VaN: $s^{\text{VaN}} \leftarrow \text{ComputeAdaptiveVaNScores}(V); \mathcal{I}_{\text{VaN}} \leftarrow \text{TopK}(s^{\text{VaN}}, (1 - \alpha)|C|)$ $\triangleright$ Sec. 3.2

 (5) Combine: $\mathcal{I} \leftarrow \mathcal{I}_{\text{TaR}} \cup \mathcal{I}_{\text{VaN}} \cup \text{Indices}(K_{\text{recent}}); K \leftarrow K[\mathcal{I}], V \leftarrow V[\mathcal{I}]$ $\triangleright$ Compress to $|C|$ size

end if

if user query arrives **then**

 Generate response using current K, V

end if

end while

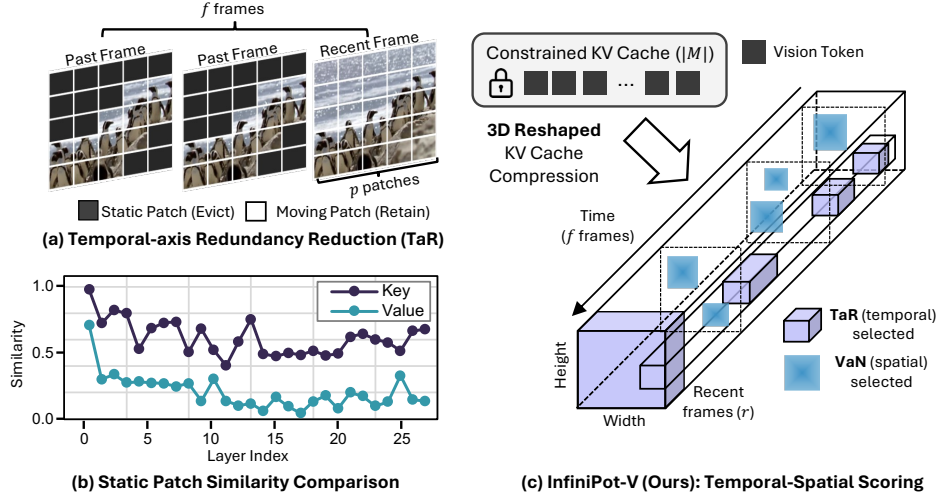


Figure 2: **Compresión espaciotemporal de KV cache (TaR y VaN).** (a) Redundancia temporal entre frames adyacentes, mostrando patches estáticos que pueden desalojarse de frames pasados; (b) similitud coseno por capa de embeddings Key/Value para patches estáticos entre frames consecutivos en LLaVA-Next-Video-7B; (c) InfiniPot-V realiza compresión espaciotemporal query-agnostic, reduciendo la redundancia temporal con TaR y seleccionando tokens mediante puntuación espacial VaN.

compresión continua de KV cache dentro de un presupuesto de memoria fijo. En este framework, los KV embeddings de los frames entrantes se almacenan hasta alcanzar el límite de memoria $|M|$. En ese punto, la compresión reduce la cache a un tamaño objetivo menor $|C|$ ($|M| \gg |C|$), reteniendo solo los vision tokens más esenciales según dos criterios. El espacio liberado ($|M| - |C|$) permite acomodar nuevos frames. Este proceso se repite continuamente, permitiendo un procesamiento eficiente del stream bajo restricciones estrictas de memoria. Cuando se emite una user query, el modelo responde usando la cache comprimida que resume el contexto visual de todos los frames previos. Notablemente, la compresión añade solo un 0.5% de overhead respecto al tiempo de procesamiento de los frames de entrada.

InfiniPot-V aprovecha dos criterios de desalojo de tokens: *Temporal-axis Redundancy (TaR)* y *Value Norm (VaN)* para identificar tokens cruciales al comprimir la KV cache. En las siguientes subsecciones detallamos cada criterio y, finalmente, describimos cómo combinarlos de forma efectiva.

3.1 Temporal-axis Redundancy (TaR) Reduction via Patch-wise Similarity

Video streams exhibit inherent spatiotemporal redundancy across frames [44, 36, 40]. In this section, we focus on exploiting temporal redundancy, as illustrated in Fig. 2(a) where static patches⁵ (e.g., background) persist across frames. For MLLMs processing videos with fixed memory usage, identifying this redundancy in KV caches is crucial. Our analysis in Fig. 2(b) reveals that Key embeddings effectively capture temporal redundancy, exhibiting higher cosine similarity for static patches between adjacent frames compared to Value embeddings, across all layers.

Building on this insight, we propose TaR, a technique that performs a patch-wise comparison of Key embeddings along the temporal axis to detect and reduce redundant tokens. As shown in Fig. 2(c), we introduce a 3D reshaping of Key embeddings to enable direct comparison of corresponding patches across frames. Based on this structured KV cache, the TaR implementation starts with a memory constraint of $|M|$ tokens, processing f consecutive video frames, each containing $p = |M|/f$ vision tokens. To maintain temporal continuity, we designate the r latest frames as *recent frames* and retain them in full. The older *past frames* ($f - r$ frames) are selectively compressed based on their patch-wise similarity to recent frames.

To measure the patch-wise similarity between frames, we divide the current Key embeddings $K \in \mathbb{R}^{H \times (f \times p) \times D}$ into $K_{\text{recent}} \in \mathbb{R}^{H \times r \times p \times D}$ and $K_{\text{past}} \in \mathbb{R}^{H \times (f-r) \times p \times D}$, representing the recent and past frames respectively. For each spatial coordinate (i, j) , we compute the ℓ_2 -normalized cosine similarity between recent and past frames of the same patch coordinate:

$$s^{\text{TaR}}(t, i, j) = -\frac{1}{r} \sum_{t'=1}^r \cos\left(K_{\text{past}}^{(t, i, j)}, K_{\text{recent}}^{(t', i, j)}\right). \quad (4)$$

Here, $s^{\text{TaR}}(t, i, j)$ is the importance score of the patch in t -th frame at (i, j) coordinate. The negative sign is applied so that a higher computed score indicates lower redundancy (i.e., the token is more distinctive). This ensures that tokens with less temporal similarity to recent frames are prioritized. We then select the least redundant tokens (i.e., higher score) in past frames using the Top-K operator:

$$\mathcal{I}_{\text{TaR}} = \text{TopK}(s^{\text{TaR}}, |C| - |K_{\text{recent}}|), \quad (5)$$

where $|C|$ is the target cache compression size and $|K_{\text{recent}}| = rp$ accounts for the recent frame tokens that are always retained. The compressed key-value pairs are formed by concatenating the selected key frame tokens with all recent frame tokens:

$$\tilde{K}_{\text{TaR}} = \text{Concat}(K[:, \mathcal{I}_{\text{TaR}}, :], K_{\text{recent}}), \quad \tilde{V}_{\text{TaR}} = \text{Concat}(V[:, \mathcal{I}_{\text{TaR}}, :], V_{\text{recent}}). \quad (6)$$

By fully preserving the most recent frames, we maintain complete information on rapidly changing or newly introduced content, while selectively retaining distinctive visual elements from the past.

3.2 Spatial Semantic Importance Preserving with Value Norm (VaN)

While TaR focuses on reducing temporal redundancy, VaN serves a complementary role: identifying and preserving semantically salient regions within each video frame, independent of the query. To achieve this, we employ Value embeddings (V), which inherently capture semantic information in transformer attention [41]. Specifically, we introduce Value Norm (VaN) as a metric for token-level semantic importance: $s^{\text{VaN}} = \|V^{(t, i, j)}\|_2$.

Analysis of Value Norm. We hypothesize that tokens with higher VaN contain richer semantic information, making them more valuable for video understanding. To quantify semantic importance, we project vision token representations from each layer into the vocabulary space [29] and compute the entropy of the

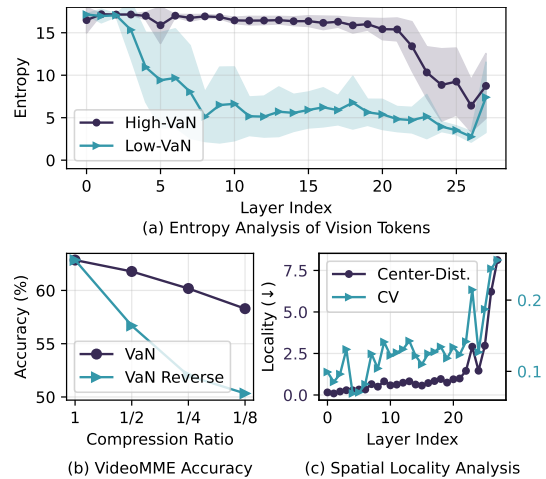


Figure 3: **Value Norm (VaN) Analysis.** (a) Entropy analysis of vision token representations grouped by their VaN scores. (b) VideoMME performance under varying cache compression ratios using either VaN or reverse-VaN for token selection. (c) Layer-wise locality of VaN, measured by center distance and coefficient of variation (CV); lower values indicate stronger spatial consistency.

⁵In MLLMs, each vision patch corresponds to a single token, so we use these terms interchangeably.

Method	Size	# Frames	Budget	EgoSchema	MLVU	VideoMME	LVB
Max Duration			$ M $	3 min	120 min	60 min	60 min
GPT4-V*	–	1fps	–	55.6	–	60.7	–
GPT4-o*	–	1fps	–	72.2	66.2	77.2	66.7
LLaVA-OV*	7B	32	8K	60.1	64.7	58.2	–
LongVU*	7B	1fps	8K	67.6	65.4	60.6	–
LongVU*	3B	1fps	8K	59.1	55.9	51.5	–
Qwen-2-VL	7B	768	50K	65.2	65.8	63.9	58.8
Qwen-2-VL + Ours	7B	768	6K	65.6	65.8	62.8	58.4
LLaVA-Next	7B	128	25K	67.6	68.7	62.8	63.5
LLaVA-Next + Ours	7B	128	6K	65.8	65.2	61.1	60.9
Qwen-2.5-VL	3B	768	50K	64.4	63.3	60.3	59.9
Qwen-2.5-VL + Ours	3B	768	6K	61.8	62.1	59.3	56.5

Table 1: Comparison of various MLLMs accuracy on four Offline Video Understanding (OVU) benchmarks. * denotes the numbers from official paper.

resulting word probability distribution, where the higher entropy implies greater informativeness [9, 3]. As shown in Fig. 3 (a), tokens with higher VaN consistently exhibit higher entropy, confirming their semantic significance. This advantage translates to improved performance: Fig. 3 (b) shows that retaining high-VaN tokens achieves substantially higher video understanding accuracy across various compression ratios compared to low-VaN (VaN Reverse) tokens.

Layer-wise Adaptive Pooling. An analysis of VaN distributions reveals strong spatial locality patterns in early to middle layers, which gradually diminish in deeper layers as shown in Fig. 3(c). To measure spatial locality patterns across layers, we employ two methods: (1) compute the average distance between the center point and surrounding points within a 3×3 window spanning the VaN values of each frame (center-dist.), and (2) measure the Coefficient of Variance (CV) to quantify dispersion of VaN distributions. Lower values in both metrics—smaller center-dist. and CV—indicate that VaN scores are closely clustered, implying high spatial locality, whereas higher values reflect greater dispersion and lower locality.

As shown in Fig. 3 (c), both metrics consistently indicate strong locality in early to middle layers, while gradually diminishing in deeper layers. Based on this observation, we design an adaptive spatial pooling mechanism that adjusts the average pooling kernel size per layer. To implement this, we design a mapping function g that assigns kernel sizes in inverse relation to each layer’s CV:

$$\text{PoolSize}(CV_l) = g(CV_l) \quad \text{where} \quad g: \mathbb{R}^+ \rightarrow 1, 3, 5, 7$$

This approach assigns larger pooling kernels (e.g., 7) to lower layers with smaller CV values (higher spatial locality), and smaller kernels (e.g., 1, implying no pooling) to upper layers with larger CV values, thus preserving fine-grained details where needed.

For KV cache compression, we select tokens using VaN scores processed through our adaptive pooling mechanism, retaining the Top- $|C|$ tokens with highest pooled VaN values as described in Fig. 2(c): $I_{\text{VaN}} = \text{TopK}(\text{VaN}_{\text{pool}}, |C|)$

$$\tilde{K}_{\text{VaN}} = K[:, I_{\text{VaN}}, :], \quad \tilde{V}_{\text{VaN}} = V[:, I_{\text{VaN}}, :]. \quad (7)$$

3.3 Design Space Exploration

Combining TaR and VaN for Token Selection. TaR and VaN capture complementary aspects of spatio-temporal redundancy in streaming video. To integrate them, we prioritize TaR-based selection by first allocating $\alpha|C|$ tokens to TaR, then filling the remaining $(1 - \alpha)|C|$ with VaN-selected tokens.

Compression	Budget	Video MME			MLVU			
Method	$ M $	Short	Med	Long	Holistic	Single	Multi	Avg.
FullKV	50K	74.7	62.1	55.0	76.3	73.9	43.3	64.2
TTC [40]	3K	66.8	51.2	47.9	72.1	58.8	33.2	54.8
(IVC)	6K	72.6	55.0	51.7	76.3	60.9	36.7	58.4
STC [36]	3K	67.9	51.0	49.3	71.5	58.6	33.9	55.0
(IVC)	6K	72.6	56.2	51.6	74.3	61.1	35.9	57.9
InfiniPot-V	3K	73.9	57.8	51.8	77.7	70.4	43.2	63.1
(CKV)	6K	74.1	60.8	53.4	77.2	72.3	44.8	64.3

Table 2: Comparison under memory-constrained settings (3, 6K memory-budget) with Input Video Compression (IVC) methods: TTC from DyCoke [40] and STC from LongVU [36]. Qwen-2-VL-7B used across VideoMME and MLVU benchmarks. Results comparing memory-unconstrained IVC methods (without KV cache compression) with InfiniPot-V are provided in Tab. A6.

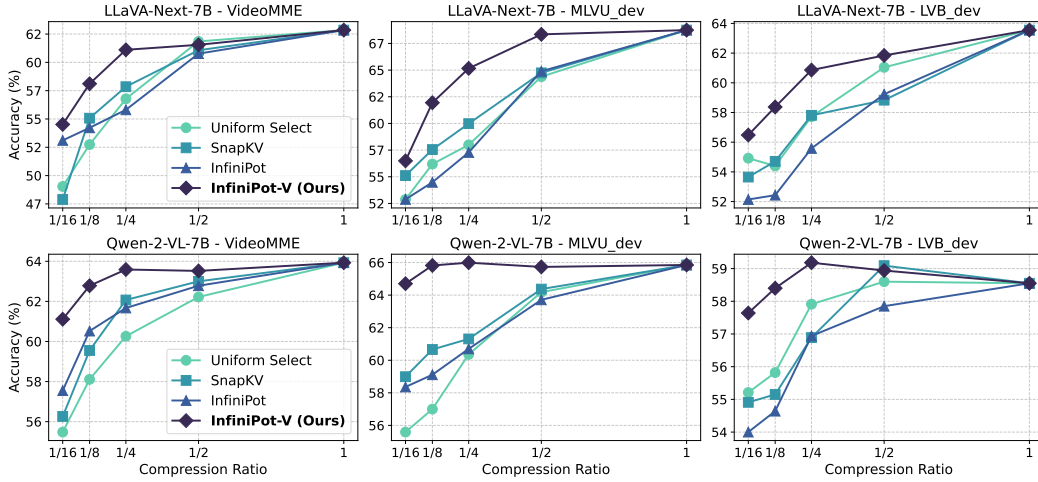


Figure 4: Resultados de evaluación de métodos de KV cache Compression (KVC) con tareas offline de long video understanding bajo el marco de Continual KV Cache Compression (CKV). Rendimiento en cuatro razones de compresión (1/16, 1/8, 1/4, 1/2) para LLaVA-Next-7B (fila superior) y Qwen-2-VL-7B (fila inferior) en las tareas VideoMME, MLVU_{dev} y LongVideoBench_{dev} (LVB_{dev}). Los resultados completos de evaluación se muestran en la Tabla A5.

This two-stage selection strategy effectively balances temporal and feature importance. A detailed hyperparameter exploration, including sweeps over α and the size of the recent frame window r , is provided in Appendix. A.2.

Comparison with Memory-Constrained Alternatives. A natural question is whether IVC or KVC can be adapted for SVU under memory constraints. To explore this, we apply query-agnostic methods such as spatial token compression (STC) and token temporal merging (TTC) from LongVU [36] and DyCoke [40]. InfiniPot-V outperforms all these baselines by a notable accuracy margin, demonstrating the strength of continual compression over expressive key-value embeddings (details in Tab. 2).

4 Experimentos

4.1 Configuración experimental

Benchmarks. Evaluamos nuestro InfiniPot-V tanto en tareas de offline video understanding (OVU) como de streaming video understanding (SVU). Para OVU, utilizamos benchmarks representativos

de long video understanding basados en múltiple opción (desde 3 minutos hasta más de 2 horas): VideoMME [11], MLVU [54], LongVideoBench (LVB) [45] y EgoSchema [28].

Para SVU, empleamos el benchmark de streaming video QA RVS-Ego/Movie [49], que presenta preguntas abiertas emparejadas con timestamps, y evaluamos las respuestas usando GPT-3.5-turbo-0125 siguiendo [49, 35]. Además extendemos la evaluación de SVU basada en múltiple opción con OVO-Bench [23] y StreamingBench [25]. OVO-Bench evalúa razonamiento temporal bajo tres escenarios—trazado retrospectivo, percepción visual en tiempo real y respuesta activa hacia adelante—mientras que StreamingBench evalúa comprensión visual en tiempo real de videos en streaming.

Modelos. Aplicamos nuestro método sobre cuatro MLLMs de estado del arte capaces de long-video understanding: Qwen-2-VL [43], Qwen-2.5-VL [48], LLaVA-OV-7B [22] y LLaVA-Next-Video [52]. Los detalles sobre las configuraciones de muestreo del video de entrada y del benchmark se proporcionan en el Apéndice. B.

4.2 Resultados de evaluación

Offline Video Understanding. Para evaluar la capacidad absoluta de compresión de nuestro método, evaluamos InfiniPot-V tanto con MLLMs comerciales (GPT-4V [30], GPT-4o [31]) como con modelos públicos de estado del arte diseñados para offline video understanding, incluyendo LLaVA-OV [22] y LongVU [36]. A diferencia de estos modelos especializados y completamente entrenados, InfiniPot-V es un framework plug-in sin entrenamiento, compatible con MLLMs de distintas escalas, que permite alto rendimiento bajo presupuestos de memoria fijos. Como se muestra en la Tab. 1, InfiniPot-V reduce el uso de memoria a solo 25% (6K tokens) para LLaVA-Next (originalmente 25K tokens) y a 12.5% para la serie Qwen-VL (6K frente a 50K), con una pérdida mínima de rendimiento. Notablemente, logra una exactitud comparable o mejor que LongVU a escala 7B y lo supera significativamente en 3B, demostrando tanto eficiencia como escalabilidad.

Comparación con IVC bajo restricciones de memoria. Para evaluar métodos IVC recientes independientes de la consulta bajo CKV restringido en memoria, adoptamos una configuración unificada sobre VideoMME y MLVU: token temporal merging (TTC) de DyCoke [40] y spatial token compression (STC) de LongVU [36] se aplican para comprimir vision tokens hasta ajustarse al presupuesto de memoria objetivo $|M|$, mientras la KV cache se gestiona usando sliding window attention (SWA) [2]. Cuando operan bajo estas restricciones, estos métodos IVC sufren una degradación notable de exactitud. En contraste, InfiniPot-V realiza compresión de KV cache usando TaR y VaN, aprovechando representaciones key-value expresivas para lograr una exactitud promedio superior bajo un presupuesto de memoria de 6K, correspondiente a una tasa de compresión sin pérdida del 88%.

Comparación con KVC bajo restricciones de memoria. La Fig.4 evalúa métodos KVC dentro de nuestro marco CKV bajo memoria restringida en tareas de offline video understanding. Las razones de compresión (1/16, 1/8, 1/4, 1/2) se definen según la capacidad máxima de frames de cada modelo (por ejemplo, 128 frames para LLaVA-Next y 768 para Qwen-2-VL). Nuestro InfiniPot-V supera consistentemente a todos los métodos baseline (Uniform Select, SnapKV, InfiniPot) en todas las tareas tanto para LLaVA-Next-7B como para Qwen-2-VL-7B, demostrando un rendimiento superior de video understanding. Bajo restricciones CKV—donde no se dispone de la consulta real—los métodos dependientes de la consulta como SnapKV[24] se degradan significativamente. En contraste, InfiniPot-V mantiene una exactitud sólida incluso con razones altas de compresión (por ejemplo, 1/16), gracias a su selección independiente de la consulta mediante TaR y VaN.

Streaming Video Understanding. Primero evaluamos InfiniPot-V en streaming video understanding (SVU) usando dos benchmarks de StreamingVQA, RVS-Ego y RVS-Movie, con LLaVA-OV-7B. Como baseline, comparamos contra ReKV [35], un método SVU de estado del arte, bajo dos configuraciones: (1) CPU-GPU con offloading a CPU, que permite volcar la KV cache a memoria CPU, y (2) CPU-GPU sin offloading, simulando dispositivos de memoria compartida donde la memoria CPU no está disponible o ya está ocupada [19]. La Tab. 3 reporta exactitud SVU, tiempo de compresión y uso de memoria. Mientras que el offloading a CPU permite a ReKV retener la KV cache completa, introduce una sobrecarga sustancial de transferencia; sin offloading, ReKV sufre

LLaVA-OV-7B	RVS-Ego		RVS-Movie		Execution Time	Total Memory Usage	
	Acc	Score	Acc	Score	Video Enc. (msec/Frame)	GPU	CPU
ReKV	60.1	3.9	53.4	3.8	285.7	37.5 GB	+ 18.8GB/h
ReKV w/o off.	55.8	3.3	50.8	3.4	74.6	27.2 GB	0
InfiniPot-V	57.9	3.5	51.4	3.5	76.3	27.8 GB	0

Table 3: Streaming benchmark comparison to offloading-based KV cache control method. (ReKV) Video Enc. shows execution time per frame, GPU indicates peak memory usage, and CPU denotes the size of video KV-Cache offloaded to CPU per hour. Results based on an 1-hour video processed with a 0.5 fps sampling rate in streaming mode. LLaVA-OV-7B is used.

Qwen-2.5-VL	Scale	OVO-BW	OVO-Real	OVO-FW	OVO-Avg	StreamingBench
Uniform Select	7B	44.5	61.1	47.8	51.7	75.2
InfiniPot-V	7B	47.6	65.9	47.9	53.6	76.4

Table 4: Comparison on OVO-Bench (BW: Backward, Real: Realtime, FW: Forward) and StreamingBench (Real-time visual understanding) using Qwen-2.5-VL-7B under 4K memory budget.

una degradación severa de exactitud debido a la memoria local limitada. En contraste, InfiniPot-V opera completamente dentro de la memoria GPU, elimina la latencia de offloading y alcanza mayor exactitud, lo que lo convierte en una solución práctica para sistemas con memoria restringida.

Para evaluar aún más la exactitud en escenarios de streaming, extendemos el análisis a dos benchmarks adicionales: OVO-Bench [23] y StreamingBench [25], como se resume en la Tab. 4. En comparación con el baseline de selección uniforme de tokens, InfiniPot-V muestra mejoras consistentes en todas las métricas, mejorando notablemente el recall de información visual pasada en la tarea OVO-BW (backward) (44.5 $\rightarrow$ 47.6) y logrando mayor exactitud de comprensión en tiempo real en OVO-Real (Realtime) (61.1 $\rightarrow$ 65.9) y en la puntuación de StreamingBench (75.2 $\rightarrow$ 76.4), demostrando la efectividad de nuestra compresión TaR-VaN para eliminar redundancia temporal mientras preserva semántica espacialmente saliente, y mostrando su capacidad bajo escenarios SVU exigentes.

4.3 Estudio de ablación

La Tab. 5 valida nuestras decisiones de diseño para TaR y VaN. Las estrategias invertidas (TaR Reverse y VaN Reverse) degradan significativamente el rendimiento al descartar tokens distintivos o semánticamente importantes. Dentro de TaR, la similitud por patch demuestra ser más efectiva que la similitud a nivel frame (64.5 vs. 62.9). VaN por sí solo supera al baseline, y su rendimiento mejora aún más con adaptive pooling (64.1 vs. 63.0). Combinar TaR y VaN produce la mayor exactitud, superando significativamente al baseline. Exploraciones adicionales de integración se discuten en el Apéndice. A.2.

4.4 Resultados de despliegue en dispositivos edge

Evaluamos nuestro marco CKV—que combina compresión continua de KV con puntuación TaR y VaN—en NVIDIA Jetson AGX Orin [19] usando Qwen-2.5-VL-3B y videos de 10 minutos de StreamingBench [25] (0.2–1 FPS). Como se muestra en la Tab. 6, nuestro método logra una eficiencia consistente en todas las métricas. La memoria pico permanece casi constante (9.2–10.7 GB) mientras que Full KV crece linealmente (13.8 $\rightarrow$ 39.0 GB), logrando una reducción de 3.6 $\times$ a los 500 segundos de video en streaming. La velocidad de Prefill, medida en FPS (frames procesados por segundo durante el prefill), se mantiene estable en 6.3–6.4 FPS, mientras que Full KV cae a 4.1 FPS. El throughput de generación también aumenta hasta 7.3 $\times$ (1.2 $\rightarrow$ 9.1 tok/sec) bajo un presupuesto fijo de memoria. El consumo de energía escala proporcionalmente con el cómputo, mejorando la eficiencia energética en casi 2 $\times$ (7.5 $\rightarrow$ 3.7 J). Notablemente, CKV permite inferencia continua en flujos de 600 segundos donde Full KV falla con OOM, confirmando su practicidad para razonamiento multimodal en tiempo real sobre dispositivos edge con memoria restringida.

MLVU _{dev}	Holistic Reasoning		Single Detail			Multi Detail		
Ablation Study	Topic	Anomaly	Plot	Needle	Ego	AO	AC	Avg
Full KV	85.2	67.5	72.7	83.9	65.1	54.1	32.5	65.9
Uniform Select	83.7	66.5	67.9	76.1	58.5	51.0	27.2	61.5
TaR Reverse	79.0	64.5	56.9	65.6	55.1	45.2	21.8	55.5
TaR Frame	82.9	66.0	67.0	78.9	63.6	51.0	31.1	62.9
TaR	85.9	66.5	71.8	78.0	62.2	51.7	35.4	64.5
VaN Reverse	78.3	66.5	56.2	66.8	53.4	46.3	17.5	55.0
VaN	84.4	68.0	68.6	76.6	61.9	52.5	29.1	63.0
VaN + Pool	85.2	68.0	71.4	77.5	63.1	52.1	31.5	64.1
TaR + VaN + Pool	86.3	68.0	72.7	80.3	63.9	54.1	35.4	65.8

Table 5: Ablation study of TaR, VaN, and their combination. Experiments conducted on MLVU using Qwen-2-VL-7B with a 6K memory budget.

Method	Length (sec)	Context Length	VRAM (GB) ↓	FPS (frame/sec) ↑	Throughput (tok/sec) ↑	Power (J) ↓
Full KV	100	18K	13.8	6.2	5.0	1.6
InfiniPot-V			9.2 (1.5×)	6.4 (1.0×)	9.2 (1.8×)	1.2 (1.4×)
Full KV	300	54K	26.6	5.0	2.0	4.7
InfiniPot-V			10.1 (2.6×)	6.4 (1.3×)	9.2 (4.6×)	2.4 (2.0×)
Full KV	500	90K	39.0	4.1	1.2	7.5
InfiniPot-V			10.7 (3.6×)	6.3 (1.5×)	9.1 (7.3×)	3.7 (2.0×)
Full KV	600	108K	OOM	OOM	OOM	OOM
InfiniPot-V			11.3	6.4	9.2	4.3

Table 6: Streaming video processing performance results on NVIDIA Jetson AGX Orin. Streaming 10-minute video samples (FPS 0.2–1) processed with Qwen-2.5-VL-3B, measuring memory, FPS (frames processed per second), throughput, and power consumption.

5 Conclusion

En este trabajo propusimos InfiniPot-V, un framework training-free de control de KV cache para procesamiento de streaming video en entornos con restricciones de memoria. Construido alrededor de restricciones prácticas—queries no disponibles y presupuestos de memoria estrictos durante la compresión—InfiniPot-V emplea dos criterios novedosos de desalojo de tokens, TaR y VaN, logrando mejoras significativas en comprensión de video largo bajo escenarios de streaming.

References

- [1] Kazi Hasan Ibn Arif, JinYi Yoon, Dimitrios S. Nikolopoulos, Hans Vandierendonck, Deepu John, and Bo Ji. Hired: Attention-guided token dropping for efficient inference of high-resolution vision-language models, 2024.
- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.
- [3] David Chan, Rodolfo Corona, Joonyong Park, Cheol Jun Cho, Yutong Bai, and Trevor Darrell. Analyzing the language of visual tokens, 2025.
- [4] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models, 2024.
- [5] Yukang Chen, Fuzhao Xue, Dacheng Li, Qinghao Hu, Ligeng Zhu, Xiuyu Li, Yunhao Fang, Hao-tian Tang, Shang Yang, Zhijian Liu, Ethan He, Hongxu Yin, Pavlo Molchanov, Jan Kautz, Linxi

- Fan, Yuke Zhu, Yao Lu, and Song Han. LongVILA: Scaling long-context visual language models for long videos. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [6] Giulio Corallo, Orion Weller, Fabio Petroni, and Paolo Papotti. Beyond rag: Task-aware kv cache compression for comprehensive knowledge reasoning, 2025.
 - [7] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
 - [8] Alessio Devoto, Yu Zhao, Simone Scardapane, and Pasquale Minervini. A simple and effective l_2 norm-based strategy for KV cache compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 18476–18499, November 2024.
 - [9] Sebastian Farquhar, Jannik Kossen, Livia Kuhn, et al. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.
 - [10] Christoph Feichtenhofer, Haoqi Fan, Jitendra Malik, and Kaiming He. Slowfast networks for video recognition, 2019.
 - [11] Chaoyou Fu, Yuhan Dai, Yondong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. *arXiv preprint arXiv:2405.21075*, 2024.
 - [12] Yu Fu, Zefan Cai, Abedelkadir Asi, Wayne Xiong, Yue Dong, and Wen Xiao. Not all heads matter: A head-level kv cache compression method with integrated retrieval and reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025.
 - [13] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *The Twelfth International Conference on Learning Representations*, 2024.
 - [14] Google DeepMind. Project ASTRA. <https://deepmind.google/technologies/project-astra/>, 2024.
 - [15] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, Miguel Martin, Tushar Nagarajan, Ilija Radosavovic, Santhosh Kumar Ramakrishnan, Fiona Ryan, Jayant Sharma, Michael Wray, Mengmeng Xu, Eric Zhongcong Xu, Chen Zhao, Siddhant Bansal, Dhruv Batra, Vincent Cartillier, Sean Crane, Tien Do, Morrie Doulaty, Akshay Erapalli, Christoph Feichtenhofer, Adriano Fragomeni, Qichen Fu, Abraham Gebreselasie, Cristina González, James Hillis, Xuhua Huang, Yifei Huang, Wenqi Jia, Weslie Khoo, Jáchym Kolář, Satwik Kottur, Anurag Kumar, Federico Landini, Chao Li, Yanghao Li, Zhenqiang Li, Karttikeya Mangalam, Raghava Modhugu, Jonathan Munro, Tullie Murrell, Takumi Nishiyasu, Will Price, Paola Ruiz, Merey Ramazanova, Leda Sari, Kiran Somasundaram, Audrey Southerland, Yusuke Sugano, Ruijie Tao, Minh Vo, Yuchen Wang, Xindi Wu, Takuma Yagi, Ziwei Zhao, Yunyi Zhu, Pablo Arbeláez, David Crandall, Dima Damen, Giovanni Maria Farinella, Christian Fuegen, Bernard Ghanem, Vamsi Krishna Ithapu, C. V. Jawahar, Hanbyul Joo, Kris Kitani, Haizhou Li, Richard Newcombe, Aude Oliva, Hyun Soo Park, James M. Rehg, Yoichi Sato, Jianbo Shi, Mike Zheng Shou, Antonio Torralba, Lorenzo Torresani, Mingfei Yan, and Jitendra Malik. Ego4d: Around the world in 3,000 hours of egocentric video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18995–19012, June 2022.
 - [16] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Monishwaran Maheswaran, June Paik, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. Squeezed attention: Accelerating long context length llm inference. *arXiv preprint arXiv:2411.09688*, 2024.
 - [17] Qingqiu Huang, Yu Xiong, Anyi Rao, Jiaze Wang, and Dahua Lin. Movienet: A holistic dataset for movie understanding. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part IV 16*, pages 709–727. Springer, 2020.

- [18] Yuxiang Huang, Binhang Yuan, Xu Han, Chaojun Xiao, and Zhiyuan Liu. Locret: Enhancing eviction in long-context llm inference with trained retaining heads on consumer-grade devices, 2025.
- [19] Leela S. Karumbunathan. NVIDIA Jetson AGX Orin Series Technical Brief. Technical Report TB_10749-001_v1.2, NVIDIA Corporation, July 2022. Version 1.2.
- [20] Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. Kvzip: Query-agnostic kv cache compression with context reconstruction, 2025.
- [21] Minsoo Kim, Kyuhong Shim, Jungwook Choi, and Simyung Chang. InfiniPot: Infinite context processing on memory-constrained LLMs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 16046–16060, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- [22] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024.
- [23] Yifei Li, Junbo Niu, Ziyang Miao, Chunjiang Ge, Yuanhang Zhou, Qihao He, Xiaoyi Dong, Haodong Duan, Shuangrui Ding, Rui Qian, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, and Jiaqi Wang. Ovo-bench: How far is your video-llms from real-world online video understanding?, 2025.
- [24] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [25] Junming Lin, Zheng Fang, Chi Chen, Zihao Wan, Fuwen Luo, Peng Li, Yang Liu, and Maosong Sun. Streamingbench: Assessing the gap for mllms to achieve streaming video understanding. *arXiv preprint arXiv:2411.03628*, 2024.
- [26] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ringattention with blockwise transformers for near-infinite context. In *The Twelfth International Conference on Learning Representations*, 2024.
- [27] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 34892–34916. Curran Associates, Inc., 2023.
- [28] Karttikeya Mangalam, Raiymbek Akshulakov, and Jitendra Malik. Egoschema: A diagnostic benchmark for very long-form video language understanding. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
- [29] Clement Neo, Luke Ong, Philip Torr, Mor Geva, David Krueger, and Fazl Barez. Towards interpreting visual information processing in vision-language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [30] OpenAI. Gpt-4v(ision) system card, 2023.
- [31] OpenAI. Hello gpt-4o. <https://openai.com/index/hello-gpt-4o/>, 2024.
- [32] Junyoung Park, Dalton Jones, Matthew J Morse, Raghavv Goel, Mingu Lee, and Chris Lott. Keydiff: Key similarity-based kv cache eviction for long-context llm inference in resource-constrained environments, 2025.
- [33] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024.

- [34] Sundar Pichai, Demis Hassabis, and Koray Kavukcuoglu. Introducing gemini 2.0: our new ai model for the agentic era. <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024>, 2024.
- [35] Zhelun Yu Shangzhe Di. Streaming video question-answering with in-context video KV-cache retrieval. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [36] Xiaoqian Shen, Yunyang Xiong, Changsheng Zhao, Lemeng Wu, Jun Chen, Chenchen Zhu, Zechun Liu, Fanyi Xiao, Balakrishnan Varadarajan, Florian Bordes, Zhuang Liu, Hu Xu, Hyunwoo J. Kim, Bilge Soran, Raghuraman Krishnamoorthi, Mohamed Elhoseiny, and Vikas Chandra. Longvu: Spatiotemporal adaptive compression for long video-language understanding, 2024.
- [37] Morgan Stanley. Humanoids: Investment implications of embodied ai. Technical report, Morgan Stanley, June 2024. Accessed via Future Management Group: <https://www.futuremanagementgroup.com/wp-content/uploads/240626-Humanoid-Robots-Morgan-Stanley.pdf>.
- [38] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [39] Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Shikuan Hong, Danning Ke, Yiwu Yao, and Gongyi Wang. Razorattention: Efficient KV cache compression through retrieval heads. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [40] Keda Tao, Can Qin, Haoxuan You, Yang Sui, and Huan Wang. Dycoket: Dynamic compression of tokens for fast video large language models, 2024.
- [41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [42] Ethan Waisberg, Joshua Ong, Mouayad Masalkhi, Nasif Zaman, Prithul Sarker, Andrew G Lee, and Alireza Tavakkoli. Meta smart glasses—large language models and the future for assistive glasses for individuals with vision impairments. *Eye*, 38(6):1036–1038, 2024.
- [43] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- [44] T. Wiegand, G.J. Sullivan, G. Bjontegaard, and A. Luthra. Overview of the h.264/avc video coding standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 13(7):560–576, 2003.
- [45] Haoning Wu, Dongxu Li, Bei Chen, and Junnan Li. Longvideobench: A benchmark for long-context interleaved video-language understanding. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- [46] Mingze Xu, Mingfei Gao, Shiyu Li, Jiasen Lu, Zhe Gan, Zhengfeng Lai, Meng Cao, Kai Kang, Yinfei Yang, and Afshin Dehghan. Slowfast-llava-1.5: A family of token-efficient video large language models for long-form video understanding, 2025.
- [47] Yuhui Xu, Zhanming Jie, Hanze Dong, Lei Wang, Xudong Lu, Aojun Zhou, Amrita Saha, Caiming Xiong, and Doyen Sahoo. Think: Thinner key cache by query-driven pruning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [48] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li,

- Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [49] Haoji Zhang, Yiqin Wang, Yansong Tang, Yong Liu, Jiashi Feng, Jifeng Dai, and Xiaojie Jin. Flash-vstream: Memory-based real-time understanding for long video streams, 2024.
 - [50] Qizhe Zhang, Aosong Cheng, Ming Lu, Zhiyong Zhuo, Minqi Wang, Jiajun Cao, Shaobo Guo, Qi She, and Shanghang Zhang. [cls] attention is all you need for training-free visual token pruning: Make vlm inference faster, 2024.
 - [51] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. Sparsevlm: Visual token sparsification for efficient vision-language model inference, 2024.
 - [52] Yuanhan Zhang, Bo Li, haotian Liu, Yong jae Lee, Liangke Gui, Di Fu, Jiashi Feng, Ziwei Liu, and Chunyuan Li. Llava-next: A strong zero-shot video understanding model, April 2024.
 - [53] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
 - [54] Junjie Zhou, Yan Shu, Bo Zhao, Boya Wu, Shitao Xiao, Xi Yang, Yongping Xiong, Bo Zhang, Tiejun Huang, and Zheng Liu. Mlvu: A comprehensive benchmark for multi-task long video understanding. *arXiv preprint arXiv:2406.04264*, 2024.

Algorithm 2 InfiniPot-V Algorithm

Require: Video stream V , memory constraint $|M|$, target KV cache size $|C|$, recent frame count r , CV thresholds $\{\tau_1, \tau_2, \tau_3\}$, TaR ratio $\alpha \in [0, 1]$, f frames corresponding to $|M|$ tokens, vision token number per single frame $p = |M|/f$

Ensure: Compressed KV cache $\{\tilde{K}_l, \tilde{V}_l\}_{l=1}^L$

- 1: Let $C_{\text{TaR}} = \alpha|C|$ be the TaR selection budget
- 2: Let $C_{\text{VaN}} = (1 - \alpha)|C|$ be the VaN selection budget
- 3: Initialize empty KV cache for each layer $l \in \{1, \dots, L\}$
- 4: **while** processing video stream V **do**
- 5: Accumulate KV embeddings until reaching $|M|$
- 6: **for** each layer l **do**
- 7: // Temporal-axis Redundancy (TaR)
- 8: Reshape K_l into $K_{\text{recent},l} \in \mathbb{R}^{H \times r \times p \times D}$ and $K_{\text{past},l} \in \mathbb{R}^{H \times (f-r) \times p \times D}$
- 9: **for** each patch (t, i, j) in past frames **do**
- 10: $s(t, i, j) = -\frac{1}{r} \sum_{t'=1}^r \cos(K_{\text{past},l}^{(t,i,j)}, K_{\text{recent},l}^{(t',i,j)})$
- 11: **end for**
- 12: $\mathcal{I}_l \leftarrow \text{TopK}(S_l, C_{\text{TaR}})$ ▷ Select least redundant tokens
- 13: // Value Norm (VaN) with Adaptive Pooling
- 14: $\text{VaN}_l \leftarrow \|V_l\|_2$
- 15: // Compute CV for adaptive pooling
- 16: $\mu_l \leftarrow \text{mean}(\text{VaN}_l)$
- 17: $\sigma_l \leftarrow \text{std}(\text{VaN}_l)$
- 18: $\text{CV}_l \leftarrow \sigma_l / \mu_l$
- 19: // Determine pooling size using mapping function g
- 20: $\text{pool_size}_l \leftarrow g(\text{CV}_l)$ ▷ Using thresholds $\{\tau_1, \tau_2, \tau_3\}$
- 21: **where** $g(\text{CV}) = \begin{cases} 7, & \text{if } \text{CV} < \tau_1 \\ 5, & \text{if } \tau_1 \leq \text{CV} < \tau_2 \\ 3, & \text{if } \tau_2 \leq \text{CV} < \tau_3 \\ 1, & \text{if } \text{CV} \geq \tau_3 \end{cases}$
- 22: $\text{VaN}_{\text{pool},l} \leftarrow \text{AveragePool2d}(\text{VaN}_l, \text{pool_size}_l)$
- 23: // Combine TaR and VaN by prioritizing TaR-selected tokens
- 24: $\text{VaN}_{\text{pool},l}[\mathcal{I}_l] \leftarrow \max(\text{VaN}_{\text{pool},l})$ ▷ Prioritize TaR tokens
- 25: $\mathcal{J}_l \leftarrow \text{TopK}(\text{VaN}_{\text{pool},l}, |C|)$ ▷ Final token selection
- 26: $\tilde{K}_l \leftarrow K_l[:, \mathcal{J}_l, :]$, $\tilde{V}_l \leftarrow V_l[:, \mathcal{J}_l, :]$ ▷ Update layer KV cache with compressed KV cache
- 27: **end for**
- 28: **end while**

A InfiniPot-V Algorithm and Configuration

A.1 Algorithm Description

Algorithm 2 presents the complete process of InfiniPot-V’s cache control framework along with its compression formulation. InfiniPot-V processes video streams by continuously pre-filling and compressing the KV cache using two token selection strategies: Temporal-axis Redundancy (TaR) and Value Norm (VaN). For TaR, the algorithm splits video frames into recent frames (the latest r frames) and past frames, then computes cosine similarities between corresponding patches to identify and remove redundant visual tokens. (Line 10)

For spatial semantic importance token selection, a layer-wise adaptive pooling mechanism based on VaN is employed. The pooling size is dynamically determined by the Coefficient of Variation (CV) of the VaN, (Line 18) where a higher CV indicates a sparser or more distinct feature distribution. Pre-computed model-specific CV thresholds $\{\tau_1, \tau_2, \tau_3\}$ determine pooling sizes from the set $\{1, 3, 5, 7\}$, selecting larger windows for uniform (low CV) VaN distributions and smaller ones for sparse (high CV) VaN distributions (Line 21).

To integrate both criteria, TaR-selected tokens are prioritized by assigning them the maximum VaN score before the final token selection. Specifically, by setting $\text{VaN}_{\text{pool},l}[\mathcal{I}_l] = \max(\text{VaN}_{\text{pool},l})$ (Line 24) and then applying a TopK selection, the algorithm ensures that temporally distinctive tokens are preserved while allowing VaN to select additional tokens based on semantic importance.

VideoMME	$ M = \alpha \text{TaR} + (1 - \alpha) \text{VaN} , M = 6\text{K}$					
Qwen-2-VL-7B	$\alpha = 0$ (VaN)	$\alpha = 0.2$	$\alpha = 0.4$	$\alpha = 0.6$	$\alpha = 0.8$	$\alpha = 1$ (TaR)
Short (-3 min)	74.4	74.3	74.1	74.9	74.4	73.8
Medium (3 - 30 min)	59.9	59.3	61.3	61.4	61.2	58.2
Long (30 - 120 min)	51.9	51.0	52.4	53.1	53.4	53.2
Average	62.1	61.6	62.6	63.1	<u>63.0</u>	61.7
LLaVA-Next-7B	$\alpha = 0$ (VaN)	$\alpha = 0.2$	$\alpha = 0.4$	$\alpha = 0.6$	$\alpha = 0.8$	$\alpha = 1$ (TaR)
Short (-3 min)	69.8	69.8	72.0	71.1	71.9	68.8
Medium (3 - 30 min)	59.3	59.3	59.2	58.7	57.7	57.3
Long (30 - 120 min)	52.1	52.1	52.7	52.0	51.4	51.0
Average	60.4	60.4	61.3	<u>60.6</u>	60.3	59.0

Table A1: **TaR and VaN Combination Ratio:** Sweep over combination ratio α in TaR and VaN combination under a 6K memory budget ($|M|$) on VideoMME. $\alpha=0$ and $\alpha=1$ correspond to VaN-only and TaR-only, respectively. The best-performing configurations are shown in **bold**, while the second-best results are underlined.

Qwen-2-VL-7B	MLVU				VideoMME			
$ M = 6\text{K}$	Holistic	Single	Multi	Avg.	Short	Medium	Long	Avg.
$r = f \times 0.125$	77.6	66.2	43.9	<u>63.1</u>	68.7	57.3	51.0	59.0
$r = f \times 0.25$	77.8	67.1	43.5	63.4	68.0	57.2	51.1	<u>58.8</u>
$r = f \times 0.50$	78.6	64.6	39.0	61.3	65.1	56.7	49.7	57.2
$ M / C = 0.75$	78.2	71.7	44.6	65.8	74.4	59.9	51.9	<u>62.1</u>
$ M / C = 0.50$	76.8	68.2	42.3	<u>63.3</u>	74.1	60.3	52.9	62.4
$ M / C = 0.25$	73.2	65.9	39.1	60.3	73.6	56.9	50.4	60.3

Table A2: **Recent Frame and Compression Ratio Exploration: Top:** Sweep over recent frame numbers r determined by multiplying various ratios (0.125, 0.25, 0.5) to f (frame number corresponding to memory budget $|M|$) in TaR. **Bottom:** Performance under varying compression ratios $|M|/|C|$ across MLVU and VideoMME with Qwen-2-VL-7B. TaR performs best with $r \leq 0.25$ and compression ratio ≥ 0.5 . The highest values are shown in **bold**, with the second-highest underlined.

A.2 Hyper-Parameter Exploration

InfiniPot-V involves three main hyper-parameters: the TaR and VaN budget allocation ratio α , the number of recent frames r used in TaR, and the target compression size C applied at each continual KV cache compression step. This section presents comparative experiments exploring each hyper-parameter.

TaR and VaN Budget Ratio (α) We compare the accuracy of offline video understanding (OVU) task across different values of α , which determines the budget allocation between TaR and VaN under a fixed memory budget ($|M| = 6\text{K}$), for both Qwen-2-VL-7B and LLaVA-Next-7B models. As shown in Tab. A1, performance peaks when α is between 0.4 and 0.6, outperforming the use of either VaN-only ($\alpha = 0$) or TaR-only ($\alpha = 1$). This confirms the effectiveness of our approach, which jointly considers both spatial and temporal dimensions for KV cache compression.

Recent Frames (r) and Compression Ratio ($|M|/|C|$) Tab. A2 presents exploration experiments for two key hyperparameters: the recent frames number r , which determines the proportion of recent frames within the memory budget in TaR, and the compression ratio $|M|/|C|$, which defines what proportion of the compression size $|C|$ to maintain relative to the memory budget $|M|$ in continual KV cache compression (CKV).

For the recent frame number r (Tab. A2, Top), we observe optimal performance on both MLVU and VideoMME benchmarks when $r \leq 0.25f$. Setting $r = 0.5f$ results in an excessive number of frames being designated as the latest frames for temporal redundancy measurement, which limits the effectiveness of redundancy reduction. This limitation is reflected in the decreased performance

metrics. (61.3 vs 63.4 in MLVU and 57.2 vs 59.0 in VideoMME) Note that the r sweep experiments are conducted using TaR-only settings ($\alpha = 1$).

For the compression ratio ($|M|/|C|$), we conduct comparative experiments across three ratios (0.75, 0.50, and 0.25). As shown in Tab. A2 Bottom, an excessive compression ratio such as 0.25 in CKV results in noticeable performance degradation. These findings confirm that a ratio of 0.5 or higher represents an appropriate configuration for CKV.

Based on these explorations, we standardize the hyperparameter values at $\alpha = 0.5$, $r = 0.125$, and $|M|/|C| = 0.75$ for all main experimental results when evaluating InfiniPot-V.

B Experimental Setting Details

B.1 MLLMs Video Sampling Details.

For all benchmarks, we employ a consistent uniform frame sampling strategy to ensure maximized long video understanding performance across all settings. For Qwen-2-VL [43], which supports dynamic image resizing based on the number of frames, we use the hyper-parameter configuration reported to yield the best performance in their original work: $\text{FPS_MAX_FRAMES} = 768$, $\text{VIDEO_MIN_PIXEL} = 128 \times 28 \times 28$ and $\text{VIDEO_MAX_PIXEL} = 768 \times 28 \times 28$. Although theoretically larger token budgets could be set, we adopt this configuration to match the optimal context length of 50K as reported in the original paper [43], on top of which we applied KV cache compression. For LLaVA, we set the number of sampled frames to 128 to ensure it remained within the model’s trained context length (<32K). With this video sampling configuration, Qwen-2-VL [43] uses 384 frames with 130 tokens per frame, resulting in a total context length of 49,920 tokens, while LLaVA-Next [52] and LLaVA-OV [22] use 128 frames with 196 tokens per frame, yielding a total of 25,088 for offline long video inputs.

B.2 Long Video Understanding Benchmark Details

Offline Video Understanding(OVU) We evaluate our method on four multiple-choice based offline video question answering benchmarks: Video-MME [11], MLVU [54], EgoSchema [28], and LongVideoBench [45]. For MLVU and EgoSchema, we use the development sets for evaluation.

For Video-MME, we report results without subtitles version. This is because prepending subtitles for all video frames as a single context block directly before the question represents an unrealistic setting that is incompatible with streaming scenarios, where subtitles are typically unavailable during real-time video processing and would not be accessible as complete context in advance.

Streaming Video Understanding (SVU) For SVU evaluation, we use two benchmarks: RVS-Ego and EVS-Movie [49]. RVS-Ego is constructed from 10 videos from the Ego4D [15] dataset, while RVS-Movie uses 22 long videos from MovieNet [17]. Each benchmark consists of a QA set containing open-ended generation questions and their corresponding timestamps indicating when each question should be presented during video streaming.

The evaluation process works as follows: during CKV processing, when the video stream reaches the timestamp of a given question sample, we present the question and generate an answer based on the compressed KV cache accumulated up to that point. The generated answers are then compared against ground-truth answers using GPT-3.5-turbo-0125 to produce accuracy and score metrics.

B.3 Baseline Settings

Input Video Compression (IVC) Details. For the comparison with Input Video Compression (IVC) methods in Tab. 2 and A6, we implement LongVU [36] and DyCoke [40] as follows: For **LongVU**, we apply Spatial Token Compression (STC) every 8 frames as specified in the original paper. STC compresses vision token embeddings by identifying temporally redundant patches using cosine similarity between patches. We adjust the similarity threshold to control the compression rate while maintaining the original methodology. For **DyCoke**, we implement Token Temporal Merging (TTM) which, similar to LongVU, compresses vision encoder output features. TTM calculates cosine similarity between patches in adjacent frames to eliminate redundant patch embeddings. Following

the original paper, we process compression every 4 frames and adjust the similarity threshold to control compression rates.

For fair comparison in the Continual KV cache compression (CKV) framework in Tab. 2, we adapt both methods to work within memory constraint $|M|$. Specifically, we compress each input video stream to size $|C|$ and implement sliding window attention [2] to evict older KV cache entries once the cache size reaches the predefined memory limit ($|M|$). This adaptation ensures all methods operate under identical memory constraints for a fair comparison with InfiniPot-V. For benchmark comparing InfiniPot-V with IVC methods that use full vision encoding without cache compression, see Tab. A6.

KV Cache Compression (KVC) Details. In Fig. 2, we compare three KV cache compression methods within Continual KV cache compression (CKV). First, **Uniform Select**, inspired by uniform video sampling approaches, selects frames at regular intervals and retains all KV cache tokens corresponding to those frames. For **SnapKV** [24], we follow the original method configuration under the CKV process, using the last 32 tokens of the given budget $|M|$ tokens as an observation window (w) to calculate attention scores for token selection (see Eq. 1 in Appendix. D.1). Additionally, we apply 1D pooling with a kernel size of 7 to these scores, as done in the original implementation⁶. For **InfiniPot** [21], we design a proxy prompt for video compression: "Provide a detailed description of this video." This prompt is utilized in the CaP method to generate attention scores and apply KV cache compression. Detailed experimental results are provided in Tab. A5.

FastV Hyper-Parameter Settings. To provide additional performance comparison with compression methods specialized for MLLMs, we also include performance comparisons with FastV [4] in Tab. A5. FastV requires two hyper-parameters, L and R , which specify the layer where token pruning begins and the percentage of tokens to prune. For a fair comparison, we adjust the R of FastV to ensure that the total number of KV cache entries across layers matches the total entry count of other baselines that maintain the same number of KV-cache entries across each layer. Specifically, for Qwen-2-VL, the (L, R) pairs corresponding to memory budgets of 3K, and 6K are set to (2, 2.8%) and (2, 5.8%) respectively.

B.4 Positional Encoding Details.

MLLM backbone LLMs utilize positional encoding to differentiate vision token positions. LLaVA-Next [52] and LLaVA-OV [22] use standard 1D RoPE [38], while Qwen-2-VL [43] employs 3D RoPE for multimodal encoding. For Offline Video Understanding (OVU), we apply KV cache compression after positional encoding (i.e., Post RoPE). However, Streaming Video Understanding (SVU) presents a challenge: continuous video stream processing can exceed the model’s maximum positional range. For example, in LLaVA models with 196 tokens per frame, streaming more than 6 minutes of video at 0.5 FPS exceeds the 32K context window (note that RVS-Ego and RVS-Movie average over 60 minutes).

To address this, we adopt strategies from InfiniPot [21] and ReKV [35], re-ordering positional indices to fit within the memory budget $|M|$ at each CKV step. Specifically, we cache the pre-positional encoded KV’s hidden states and re-assign positional indices during decoding, ensuring they never exceed $|M|$ position regardless of video length. While this enables SVU for arbitrarily long videos, it discards the original positional information of vision tokens. In particular, additional handling is required for Qwen-2-VL’s 3D RoPE. Developing methods that preserve the original spatial and temporal position encoding while supporting streaming video lengths beyond the model’s positional capacity remains an open direction for future work.

C Multi-Turn Video Understanding Analysis

Fig. A1 presents a qualitative comparison between query-dependent (SnapKV)[24] and query-agnostic (InfiniPot-V) KV cache compression approaches in multi-turn conversations with streaming video input. When SnapKV performs compression based on Q1, it generates answers almost identical to the Full KV cache for that specific query (Q1), answering that the butter was placed in the refrigerator.

⁶<https://github.com/FasterDecoding/SnapKV>

Q1: Where did I put the butter after eating? [Single-Detail]		
Full-KV:	After eating, the person put the butter in the refrigerator. 🧡	
SnapKV:	The person put the butter in the fridge after eating. 🧡	
InfiniPot-V:	The person in the video put the butter in the fridge after eating. 🧡	
Q2: When turning off the stove, what kind of kitchen utensil were on the stove? [Single-Detail]		
Full-KV:	There was a kettle on the stove. 🧡	
SnapKV:	There were two spoons on the stove. ❌	
InfiniPot-V:	There was a kettle on the stove. 🧡	
Q3: How many pieces of bread appear in total during the cooking process? [Multi-Detail]		
Full-KV:	The video shows that there are two pieces of bread in the pan and two pieces of bread on the plate, total of 4 pieces of bread. 🧡	
SnapKV:	The video shows that there are two pieces of bread in total during the cooking process. ❌	
InfiniPot-V:	The video shows a total of 4 pieces of bread being used during the cooking process. 🧡	

Figure A1: **Qualitative Results of Multi-Turn Conversation:** Full-KV uses 16K cache while InfiniPot-V and SnapKV employ 3K compressed KV cache. SnapKV performs query-guided cache compression based on Q1 before proceeding with multi-turn conversation. The video sample is from the MLVU ego reasoning subtask, using the Qwen-2-VL-7B model. 128 frame sampling is used.

However, this query-guided compression strategy reveals significant limitations when handling different types of queries (Q2, Q3) about the same video content. Specifically, SnapKV makes critical errors in subsequent queries - misidentifying a kettle as "two spoons" in Q2 and incorrectly counting the total number of bread pieces in Q3.

In contrast, InfiniPot-V maintains accurate answers consistently across all three queries using the same 3K compressed KV cache. It correctly identifies that the butter was placed in the fridge (Q1), recognizes the kettle on the stove (Q2), and counts all 4 pieces of bread throughout the cooking process (Q3), demonstrating the effectiveness of query-agnostic compression for multi-turn streaming video scenarios.

D Why Query-Agnostic KV Cache Compression Matters for SVU?

In this section, we provide a detailed analysis of why query-agnostic compression is essential for Streaming Video Understanding (SVU), building upon the requirements discussed in Sec. 2. To demonstrate how these SVU-specific constraints impact existing KV cache compression methods, we present a case study across three representative scenarios.

D.1 Preliminary: Attention-based KV Cache Compression

Eviction-based KV cache compression reduces cache size by removing tokens with the lowest importance scores. Employing attention scores for computing token importance scores is the predominant approach in previous methods [24, 4, 12, 16].

In methods such as SnapKV [24], the importance scores u_t of a token x_t are computed by aggregating attention scores from the last w tokens (i.e., observation window) which contain the user instruction:

$$u_t = \sum_{i=N-w}^N \text{Attn}(x_i \rightarrow x_t), \quad (1)$$

where N is the current sequence length. Using these scores, the KV cache is compressed by retaining the top- M tokens with the highest aggregated attention scores. Here, M defines the memory budget: $\mathcal{I} = \text{TopK}(u, M)$ and $u = [u_1, \dots, u_N]$ indicates the importance scores of all tokens. The compressed Key and Value caches are then formed by extracting tokens at indices $\mathcal{I}$:

$$\tilde{K} = K[:, \mathcal{I}, :], \quad \tilde{V} = V[:, \mathcal{I}, :] \quad (2)$$

where $K, V \in \mathbb{R}^{H \times N \times D}$ are the uncompressed Key and Value caches with H heads, N tokens, and per-head dimension D . This approach has two characteristics: (1) it requires computing the full KV cache for all tokens before compression, and (2) it requires the user query to be present at the end

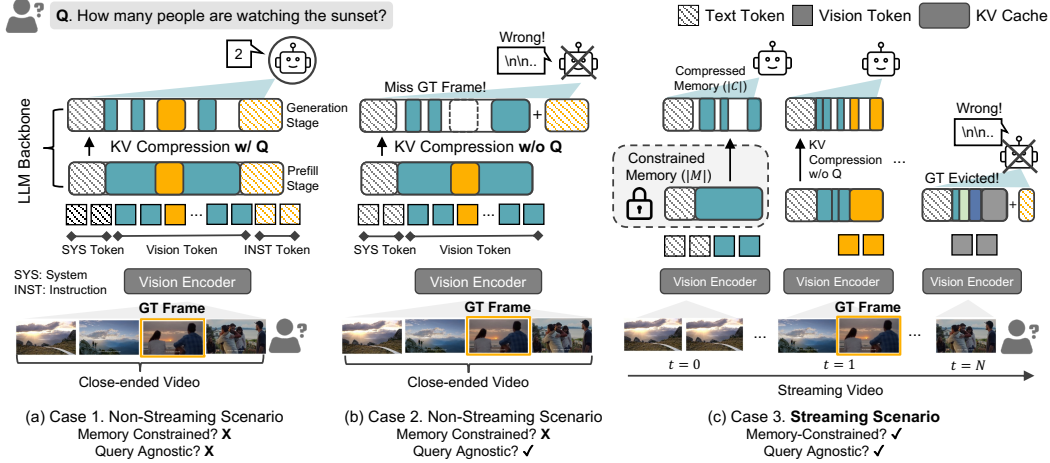


Figure A2: **KV Cache Compression Case Study with SVU**: Illustration of cache control strategies under three conditions, differing in the presence of two core requirements for Streaming Video Understanding (SVU): memory constrained (MC) and query agnostic (QA). (a) Case 1: Query-guided compression retains relevant (GT) frames for accurate responses. (b) Case 2: Without query guidance, compression fails to preserve critical frames, resulting in inaccurate responses. (c) **Case 3 (Streaming scenario)**: In streaming video processing, where frames arrive continuously, continual KV cache compression (CKV) is necessary, but queries are unavailable during compression.

of the context. We refer to these approaches as *query-guided* or *attention-based* cache compression methods.⁷

D.2 Case Study: Towards Streaming Video Understanding with CKV

To investigate the applicability of attention-based KV cache compression methods to streaming video understanding, we examine three cache control strategies (Fig. A2).

Case 1. Recent KV cache compression methods [24, 12] assume full access to context and queries at compression time, as shown in Fig. A2(a). In this memory-unconstrained setting, the model observes the full input before compression. Previous works [24] have demonstrated that attention scores effectively identify query-relevant tokens KV cache (orange box corresponding to GT Frame in Fig. A2(a)), enabling compression that retains critical information while discarding less important tokens. As shown in Tab. A3, this approach maintains performance comparable to the uncompressed cache setup (68.01 vs 68.75) at the cost of large memory usage at compression, detailed in Fig. 1.

Case	x_t Attention Scoring	Prefill $ M $	Gen. $ M = 3K$	Gen. $ M = 6K$
Full KV	n/a	25K	68.75 (↑)	
Case 1	$\text{Attn}(q \rightarrow x_t)$	25K	68.01	68.40
Case 2	$\text{Attn}(q' \rightarrow x_t)$	25K	60.35	63.42
	$\text{Attn}(q'' \rightarrow x_t)$		60.60	63.50
	$\text{Attn}(q_v \rightarrow x_t)$		60.32	62.28
Case 3	$\text{Attn}(q_v \rightarrow x_t)$	3K/6K	57.55	59.98

Table A3: **Case study of Attention Scoring**: conducted on MLVU benchmark with LLaVA-Next-Video-7B. Note that memory-constrained setting (Case 3) shares the same budget during prefill and generation stages.

Case 2. Fig. A2(b) illustrates how attention-based cache compression fails when user queries are unavailable during compression. Under this scenario, although the memory budget is assumed unconstrained, the KV cache is compressed without consideration of (future) queries, causing important visual tokens (orange tokens cache corresponding to the GT Frame) lost during compression. To quantify this degradation and explore alternatives, we test compression with generic queries (q' : "What is happening in this video?", q'' : "What are the key events in this video?") and the last vision tokens (q_v) for importance scoring:

⁷Throughout this paper, "query" refers to the user's instruction or question related to the given video.

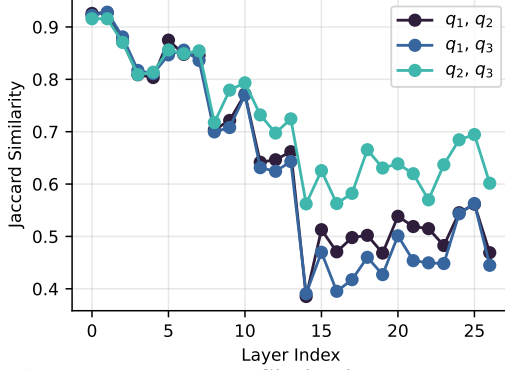


Figure A3: **Jaccard Similarity between KV Caches:** Compare KV cache sets selected by different queries (q_1, q_2, q_3) across layers.

Context Length	Case 1, 2		Case 3 (Ours)	
	Mem (GB)	TTFT (s)	Mem (GB)	TTFT (s)
5K	21.29	0.98	20.93	1.08
25K	33.76	1.21	21.60	1.12
50K	58.55	2.12	22.16	1.17
100K	79.38	3.27	22.85	1.20

Table A4: **Peak GPU Memory and TTFT:** Comparison of peak memory usage and Time-To-First-Token(TTFT) across different context lengths for memory-unconstrained (Case 1, 2) and memory-constrained (Case 3) approaches.

$$u_t^{\text{alt}} = \text{Attn}(q_{\text{alt}} \rightarrow x_t), \quad q_{\text{alt}} \in \{q', q'', q_v\} \quad (3)$$

Tab. A3 and Fig. A2(a) show that these alternatives significantly degrade performance (60.32 vs 68.75), even with unconstrained memory.

Case 3: Streaming Scenario. Beyond the query-agnostic challenge in Case 2, deploying streaming video understanding on resource-constrained devices requires fixed memory usage for the KV cache. For input video streams, these constraints necessitate continual compression when new frames arrive and memory capacity is reached, as shown in Fig. A2(c). To evaluate this scenario, we use the query-agnostic approach from Case 2 with vision tokens (q_v) for importance scoring, while compressing the KV cache whenever memory limits are reached. As shown in Tab. A3, this combined constraint further degrades performance (57.55 vs 60.32), highlighting the challenge of preserving key information under both query-agnostic and memory-constrained settings.

This case study reveals two key challenges for KV cache compression in streaming video: 1) the need for *query-agnostic compression* due to continuous incoming video, and 2) the requirement to maintain *fixed memory constraints*. These challenges cause significant performance drops in previous methods [24, 21, 4], motivating Continual KV cache compression (CKV) specifically designed for memory-constrained streaming video.

Attention Scoring Analysis We further analyze the query-dependent nature of attention-based KV cache compression using the VideoMME benchmark. To investigate why performance varies with different queries, we compute the Jaccard similarity between token sets selected for different queries across layers using attention scores at each layer. For this analysis, q_1, q_2, q_3 represent three distinct questions associated with the same video sample in the VideoMME benchmark. As shown in Fig. A3, the similarity between token sets decreases significantly in the middle-to-late layers, dropping to around 0.4. This indicates that each query selects a different set of tokens, particularly in deeper layers. This analysis highlights that attention-based scoring methods inherently select query-specific tokens, explaining the performance degradation when query information is unavailable or changes during streaming video scenarios.

E Memory and Latency Measurement Results

Table A4 presents measurements of peak memory consumption and Time-To-First-Token (TTFT) during the prefill stage, conducted on a single NVIDIA A100-80GB GPU using PyTorch. The experiments averaged over five runs with three warmup iterations, compare the performance of memory-unconstrained (Case 1, 2) and memory-constrained (Case 3) approaches across various context lengths. For memory-unconstrained methods, we observe a linear growth in memory requirements, escalating from 21.29 GB at 5K tokens to 79.38 GB at 100K tokens, accompanied by a proportional increase in TTFT from 0.98 to 3.27 seconds.

Our memory-constrained continual KV cache compression (Case 3) exhibits remarkably different behavior. Despite the increasing context length, the peak memory usage shows only minimal growth, rising modestly from 20.93 GB at 5K tokens to 22.85 GB at 100K tokens. Similarly, the TTFT remains relatively stable, increasing from 1.08 to 1.20 seconds across the same range. These detailed measurements demonstrate that our approach effectively maintains near-constant resource utilization while processing extended video frames.

F Trabajo relacionado

F.1 MLLMs para Long Video Understanding

Los avances recientes en MLLMs de contexto largo han atraído una atención significativa. Entre los ejemplos destacados se encuentran Gemini-2.0 [34], con soporte para streaming video; LongVILA [5], capaz de manejar hasta 6,000 frames de video; LLaVA-Next-Video [52], que aprovecha datos sintéticos de instrucciones de alta calidad; y Qwen-2-VL [43], que permite análisis de videos de una hora mediante multimodal RoPE.

F.2 Input-Vision Compression (IVC)

Para abordar las demandas computacionales del procesamiento de videos de larga duración, se han propuesto varios enfoques para comprimir información visual redundante antes de que ingrese al backbone LLM.

LongVU [36] adopta muestreo de frames de entrada dependiente de la consulta y eliminación de píxeles redundantes para fine-grained video understanding, pero el vision encoding de dos torres produce alta latencia durante el muestreo de entrada, lo que lo vuelve impráctico para escenarios de streaming. Además, este enfoque requiere entrenar modelos especializados para operar de la manera propuesta, limitando su aplicabilidad a modelos preentrenados existentes.

DyCoke [40] reduce redundancias entre frames adyacentes a nivel del video de entrada y actualiza dinámicamente tokens relacionados con la consulta en la KV cache desde almacenamiento externo. Slow-Fast-LLaVA-1.5 [46] propone dividir el procesamiento del video de entrada en trayectorias slow y fast separadas, usando distintos métodos de proyección para reducir los input vision tokens. Sin embargo, este enfoque todavía sufre la limitación de requerir que todos los input vision tokens se procesen simultáneamente y necesita entrenamiento adicional del modelo.

F.3 KV Cache Compression (KVC)

Comprender el contexto largo en MLLMs exige un control eficiente de la KV cache para manejar el crecimiento de memoria y la sobrecarga de latencia. Los métodos de compresión de KV cache pueden clasificarse, en términos generales, en enfoques dependientes e independientes de la consulta.

Compresión de KV cache dependiente de la consulta. Métodos como SnapKV [24], H2O [53], HeadKV [12] y ThinK [47] aprovechan puntajes de atención consulta-contexto para identificar entradas KV cruciales, pero requieren que el contexto completo pase por prefill antes de la compresión, lo que los vuelve imprácticos bajo restricciones de memoria. En el dominio multimodal, FastV [4] acelera el prefill podando vision tokens en ciertas capas según sus puntajes de atención desde el token final de consulta. SparseVLM [51] selecciona visual tokens relevantes para las consultas del usuario mediante cross-attention. En general, los métodos dependientes de la consulta comprimen eficazmente el contexto, pero tienen dificultades para manejar consultas diversas sobre un contexto ya comprimido [39]. ReKV [35] aborda escenarios de streaming video descargando la KV cache relacionada con video a memoria CPU y recuperando bajo demanda entradas de cache dependientes de la consulta. Este enfoque depende de almacenamiento externo y sufre sobrecarga de transferencia de datos, lo que lo vuelve inadecuado para streaming video understanding con memoria restringida.

Compresión de KV cache independiente de la consulta. Trabajos recientes persiguen la compresión de KV cache independiente de la consulta para eliminar la dependencia de consultas futuras [13, 8, 16, 20, 6, 32]. En particular, SqueezedAttention [16] usa clustering basado en keys, pero requiere encoding de contexto completo, lo que limita su aplicabilidad en escenarios con memoria

Case	Streaming MC	QA	Compression Method	Prefill Budget	Decoding Budget	VideoMME				MLVU				LVB	Avg.
						Short	Medium	Long	Avg.	Holistic	Single	Multi	Avg.		
Qwen-2-VL-7B															
Case 1	-	-	Full KV	50K	50K	74.68	62.11	55.00	63.93	76.34	73.91	43.29	65.85	58.77	62.85
	✗	✗	FastV [4] ($L = 2$)	48/3K ($R = 2.8$) 48/6K ($R = 5.8$)		54.11 59.67	50.11 54.55	48.67 50.78	50.96 55.00	69.59 72.00	59.40 64.08	33.84 33.47	55.01 57.60	47.94 50.53	51.30 54.38
			SnapKV [24]	50K 50K	3K 6K	74.00 74.22	61.00 60.55	54.22 54.33	63.07 63.03	77.08 77.59	67.49 73.91	39.07 42.90	62.11 66.10	59.06 58.80	61.42 62.64
Case 2	✗	✓	Uniform	50K 50K	3K 6K	70.33 72.00	54.67 58.78	49.55 52.11	58.18 60.96	72.29 77.08	59.06 67.49	33.51 39.07	55.54 62.11	59.80 59.11	57.84 60.73
			SnapKV [†]	50K 50K	3K 6K	69.00 72.11	54.00 57.56	50.67 52.22	57.89 60.63	75.88 76.46	63.48 66.43	35.35 36.22	58.99 60.66	56.70 56.72	57.86 59.34
Case 3 (CKV)	✓	✓	Uniform	3K	3K	66.00	52.44	48.00	55.48	72.54	59.00	33.51	55.59	55.21	55.43
				6K	6K	72.33	53.33	48.67	58.11	72.55	62.19	33.67	57.00	55.82	56.98
				12K	12K	74.00	55.33	51.44	60.26	75.94	65.53	37.01	60.36	57.91	59.51
			SnapKV [†]	24K	24K	74.22	59.22	53.22	62.22	77.22	71.10	40.78	64.18	58.60	61.67
				3K	3K	66.67	52.22	49.89	56.26	75.88	63.48	35.35	58.99	54.91	56.72
				6K	6K	72.00	55.33	51.33	59.55	76.46	66.43	36.22	60.66	55.15	58.45
InfiniPot [21]	12K	12K	74.44	58.89	52.89	62.07	75.71	68.61	35.98	61.31	56.89	60.09			
	24K	24K	74.22	61.00	53.78	63.00	77.66	71.82	39.90	64.37	59.09	62.15			
	3K	3K	67.11	54.55	51.00	57.55	74.94	61.80	36.60	58.36	54.00	56.64			
6K	6K	72.89	57.33	51.33	60.52	75.02	63.18	37.09	59.11	54.64	58.09				
12K	12K	74.00	57.78	53.22	61.67	74.46	66.46	38.30	60.70	56.94	59.77				
24K	24K	74.22	60.55	53.56	62.78	76.03	71.11	40.29	63.71	57.85	61.44				
		InfiniPot-V	3K	3K	73.89	57.78	51.78	61.11	77.73	70.38	43.15	64.70	57.64	61.15	
			6K	6K	74.11	60.78	53.44	62.78	77.16	72.31	44.75	65.82	58.40	62.33	
			12K	12K	74.22	62.68	53.89	63.59	76.90	73.41	43.97	65.99	59.18	62.92	
			24K	24K	74.22	63.22	53.11	63.52	76.91	73.97	42.18	65.73	58.94	62.73	
LLaVA-Next-Video-7B															
	-	-	Full KV	25K	25K	74.33	60.11	54.11	62.85	80.60	73.73	49.43	68.75	63.55	65.05
Case 1	✗	✗	Uniform	25K 25K	3K 6K	74.33 73.89	62.33 62.00	55.00 54.78	63.89 63.56	80.29 80.66	72.38 72.25	49.19 49.62	68.01 68.19	62.35 62.55	64.75 64.76
			SnapKV [24]	25K 25K	3K 6K	74.44 74.44	59.89 60.11	53.78 53.78	62.70 62.78	80.41 80.60	73.01 73.45	49.67 49.48	68.46 68.64	62.34 62.34	64.50 64.59
			Uniform	25K 25K	3K 6K	66.33 71.00	54.00 56.33	49.67 51.55	56.67 59.63	75.12 77.84	59.65 65.60	38.55 43.92	58.04 62.90	59.14 61.69	57.95 61.41
Case 2	✗	✓	SnapKV [†]	25K 25K	3K 6K	64.00 69.55	54.55 58.44	51.11 52.78	56.55 60.26	78.53 80.86	59.73 63.65	41.69 45.07	59.94 63.26	56.19 59.90	57.56 61.14
			Uniform	1.5K 3K 6K 12K	1.5K 3K 6K 12K	56.22 59.22 64.89 72.67	46.89 51.55 55.67 59.89	44.00 47.44 49.78 53.00	49.04 52.74 56.78 61.85	69.72 74.30 76.71 80.03	52.53 57.25 61.14 67.33	36.53 36.48 34.55 44.31	52.87 56.19 57.99 64.38	54.92 54.40 57.72 61.04	52.28 54.44 57.50 62.42
Case 3 (CKV)	✓	✓	SnapKV [†]	1.5K	1.5K	52.40	58.00	51.33	47.89	74.92	56.89	32.62	55.11	53.65	52.22
				3K	3K	62.11	54.55	48.55	55.07	76.94	59.18	35.71	57.55	54.71	55.78
				6K	6K	66.33	56.11	51.11	57.85	79.60	62.15	37.12	59.98	57.81	58.55
			12K	12K	72.11	58.00	53.11	61.07	79.71	67.99	44.89	64.74	58.83	61.55	
			InfiniPot [21]	1.5K	1.5K	53.22	51.11	47.55	53.11	69.89	56.44	30.54	52.88	52.14	52.71
				3K	3K	58.22	51.78	49.33	54.22	72.42	55.88	34.45	54.48	52.43	53.71
6K	6K	62.44		53.89	51.11	55.81	76.46	57.97	37.07	57.28	55.58	56.22			
12K	12K	70.55	59.22	52.55	60.77	79.84	67.81	45.57	64.89	59.23	61.63				
		InfiniPot-V	1.5K	1.5K	63.89	52.55	47.11	54.52	77.08	57.32	34.64	56.49	56.48	55.83	
			3K	3K	67.78	56.22	50.33	58.11	77.88	65.74	40.31	61.94	58.37	59.47	
			6K	6K	72.44	59.55	51.33	61.11	80.03	69.41	43.93	65.16	60.86	62.38	
			12K	12K	73.89	58.67	52.11	61.55	80.91	71.16	51.57	68.35	61.84	63.91	

Table A5: **InfiniPot-V** vs **KVC** Offline long video understanding evaluation results under memory-constrained scenario (case 3), with MC (Memory-Constrained) and QA (Query-Agnostic) conditions marked. Results are reported on (1) Video-MME - Short: -3min, Medium: 3-30min, Long: 30min-2h, (2) MLVU - Holistic, Single-Detail, Multi-Detail LVU, and (3) LVB (LongVideoBenchmark).

restringida. InfiniPot [21] comprime el contexto aproximando posibles consultas del usuario mediante un proxy prompt específico de la tarea, pero su prompt fijo restringe la flexibilidad. En el dominio de visión, HiRED [1] y FasterVLM [50] utilizan puntajes de atención del token [CLS] para las decisiones de compresión. Sin embargo, su dependencia de tokens especiales restringe su aplicación a MLLMs recientes que carecen de tales tokens [48, 52], limitando su aplicabilidad más amplia.

Qwen-2-VL	Vision	Decoding	MLVU				VideoMME				
IVC Methods	Budget	Budget	Holistic	Single	Multi	Avg.	Short	Med	Long	Avg.	Avg.
Full KV	50K	50K	76.3	73.9	43.3	65.9	74.7	62.1	55.0	63.9	64.2
Uniform	50K	6K	77.7	69.8	41.6	64.0	74.9	58.0	52.8	61.9	62.5
TTM [40]	50K	6K	78.2	70.0	42.7	64.5	74.9	59.2	52.7	62.3	62.9
STC [36]	50K	6K	77.9	71.5	44.7	65.7	74.3	59.6	54.6	62.8	63.8
InfiniPot-V	6K	6K	77.2	72.3	44.7	65.8	74.1	60.8	53.4	62.8	63.8
Uniform	50K	3K	75.7	66.5	38.6	61.1	72.2	53.4	50.0	58.6	59.4
TTM [40]	50K	3K	77.3	67.8	39.5	62.4	72.7	56.2	52.2	60.4	61.0
STC [36]	50K	3K	76.9	68.2	41.7	63.1	71.2	55.9	53.7	60.3	61.3
InfiniPot-V	3K	3K	77.7	70.4	43.2	64.7	73.9	57.8	51.8	61.1	62.5

Table A6: **InfiniPot-V vs IVC**: Performance comparison between Input-Vision Compression (IVC) methodology and InfiniPot-V. Vision budget denotes the vision token length before IVC, while decoding budget refers to the input token length used during decoding. Evaluated using Qwen-2-VL with MLVU and VideoMME datasets.

G Experimental Results Data

G.1 Comparison between InfiniPot-V and KVC

Tab. A5 provides a detailed performance comparison between KV cache compression (KVC) methods and InfiniPot-V across offline video understanding (OVU) benchmarks under various compression ratios for two models: Qwen-2-VL and LLaVA-Next.

In Case 1, where the full prefill is conducted and the final query is accessible at compression time, FastV demonstrates significantly inferior performance at similar compression ratios due to its aggressive token-pruning strategy. In contrast, SnapKV shows robust performance at high compression ratios across both models by utilizing the full context KV cache and retaining vision tokens that are highly correlated with the given query.

Case 2 examines the query-agnostic setting, where, as explored in our earlier case study in Appendix. D.2, SnapKV exhibits notable performance degradation across both models when applied in a query-agnostic manner, showing performance comparable to uniform selection baseline.

In Case 3, which represents the CKV framework scenario where the constrained memory budget is used for both prefill and decoding stages, InfiniPot-V significantly outperforms all three baselines across various compression ratios on both models, as showcased in Fig. 2.

G.2 Comparison between InfiniPot-V and IVC

Table A6 presents a performance comparison between Input-Vision Compression (IVC) methods and InfiniPot-V on the MLVU and Video-MME benchmarks using the Qwen-2-VL model. Under a 6K decoding budget, the IVC methods demonstrate robust overall performance by utilizing the full vision encoding budget (50K tokens). InfiniPot-V achieves comparable or slightly superior performance to these methods while operating under constrained memory budgets for both vision encoding and decoding stages (6K tokens).

When the decoding budget is compressed to 3K tokens, the IVC methods exhibit performance degradation, with LongVU’s STC methodology achieving the highest performance among the IVC approaches. Notably, InfiniPot-V demonstrates both efficiency and effectiveness by achieving higher accuracy than IVC methods that utilize the full vision encoding budget, while operating under constrained budgets (3K) for both vision encoding and decoding stages.

H Limitation and Future Work

InfiniPot-V introduces the first training-free, query-agnostic framework for memory-constrained streaming video understanding, enabling length-independent KV cache compression with minimal accuracy loss across long-form, real-time scenarios. However, several avenues exist for further

advancement. Current approaches focus primarily on vision tokens, yet real-world streaming applications involve multiple modalities including speech, text, and video simultaneously. Future work could extend our framework to unified multimodal compression, enabling more realistic and comprehensive streaming understanding systems that efficiently manage diverse input types within fixed memory constraints.

Additionally, our current fixed budget allocation between TaR and VaN components could benefit from adaptive mechanisms that dynamically adjust compression ratios based on input characteristics—allocating more resources to temporal redundancy reduction for static scenes or prioritizing spatial importance for content-rich frames. Furthermore, while InfiniPot-V’s training-free nature ensures broad applicability, end-to-end learning approaches could optimize models specifically for continual compression scenarios, potentially enabling more aggressive compression ratios through learned token importance estimation [18] tailored to streaming video understanding tasks.